21. PYTHON PROGRAM FOR TIC TAC TOE GAME

Aim

To write a Python program to implement the Tic Tac Toe game for two players using a 3 × 3 board.

Algorithm

1. Start the program.
2. Create a 3 × 3 game board.
3. Set player X as the first player.
4. Display the board.
5. Ask the current player to enter a position from 1 to 9.
6. Check whether the position is empty.
7. Place the player's symbol in the selected position.
8. Check whether the player has won.
9. If all positions are filled without a winner, declare a draw.
10. Otherwise, switch players and repeat.
11. Stop the program.

Python Code

```
# Tic Tac Toe Game

board = [' '] * 9

def display():
    print()
    for i in range(0, 9, 3):
        print(board[i], '|', board[i+1], '|', board[i+2])
        if i < 6:
            print('--+---+--')
    print()

def check_winner(player):
    wins = [
```

```
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]

    for a, b, c in wins:
        if board[a] == board[b] == board[c] == player:
            return True
    return False

player = 'X'

while True:
    display()
    print("Player", player)

    try:
        position = int(input("Enter position (1-9): ")) - 1
    except ValueError:
        print("Enter a valid number.")
        continue

    if position < 0 or position > 8 or board[position] != ' ':
        print("Invalid position! Try again.")
        continue

    board[position] = player

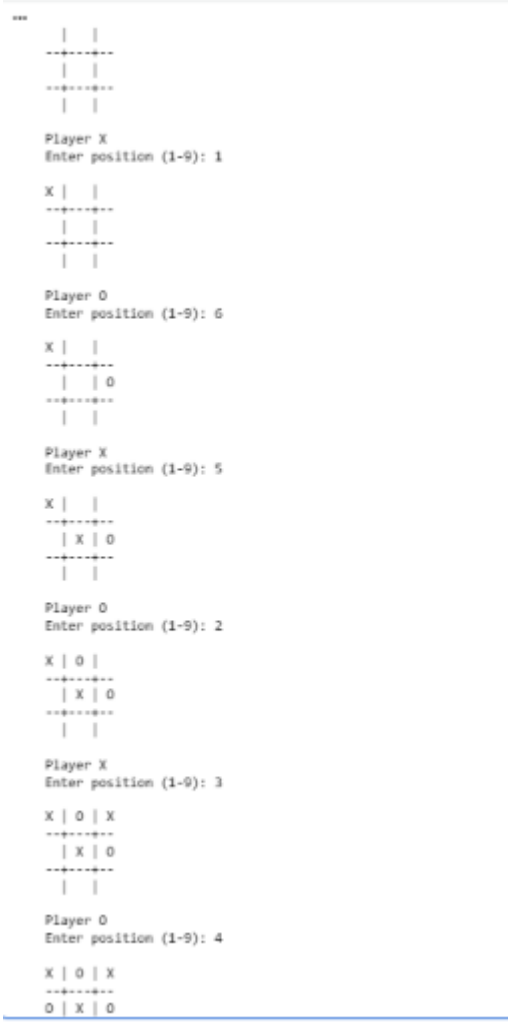
    if check_winner(player):
        display()
        print("Player", player, "wins!")
```

```
        break


    if ' ' not in board:
        display()
        print("Game Draw!")
        break


    player = 'O' if player == 'X' else 'X'
```

Output:



22. PYTHON PROGRAM TO IMPLEMENT MINIMAX ALGORITHM FOR GAMING

Aim

To write a Python program to implement the Minimax algorithm for selecting the best move in a game.

Algorithm

1. Start the program.
2. Define the game tree with possible scores.
3. Set the maximum player and minimum player.
4. The maximum player selects the highest score.
5. The minimum player selects the lowest score.
6. Repeat until the final score is reached.
7. Display the best score.
8. Stop the program.

Python Code

```
# Minimax Algorithm

def minimax(depth, node, is_max, scores):
    if depth == 2:
        return scores[node]

    if is_max:
        return max(
            minimax(depth + 1, node * 2, False, scores),
            minimax(depth + 1, node * 2 + 1, False, scores)
        )
    else:
        return min(
            minimax(depth + 1, node * 2, True, scores),
            minimax(depth + 1, node * 2 + 1, True, scores)
        )

scores = [3, 5, 2, 9]

best_score = minimax(0, 0, True, scores)
```

print("Leaf Scores:", scores)

print("Best Score using Minimax:", best_score)

Output:

```
print("Leaf Scores:", scores)
print("Best Score using Minimax:", best_score)

Leaf Scores: [3, 5, 2, 9]
Best Score using Minimax: 3
```

23. PYTHON PROGRAM TO IMPLEMENT ALPHA-BETA PRUNING ALGORITHM FOR GAMING

Aim

To write a Python program to implement Alpha-Beta Pruning to reduce the number of nodes evaluated in a game tree.

Algorithm

1. Start the program.
2. Define the game tree with leaf scores.
3. Initialize alpha to negative infinity and beta to positive infinity.
4. Find the maximum value at the maximizing node.
5. Find the minimum value at the minimizing node.
6. If alpha is greater than or equal to beta, prune the remaining branches.
7. Display the best score.
8. Stop the program.

Python Code

```
# Alpha-Beta Pruning


def alphabeta(depth, node, alpha, beta, is_max, scores):
  if depth == 2:
    return scores[node]


  if is_max:
    best = float('-inf')
```

```
        for child in [node * 2, node * 2 + 1]:
            value = alphabeta(
                depth + 1, child, alpha, beta, False, scores
            )
            best = max(best, value)
            alpha = max(alpha, best)

            if beta <= alpha:
                break

        return best

    else:
        best = float('inf')

        for child in [node * 2, node * 2 + 1]:
            value = alphabeta(
                depth + 1, child, alpha, beta, True, scores
            )
            best = min(best, value)
            beta = min(beta, best)

            if beta <= alpha:
                break

        return best

scores = [3, 5, 2, 9]

best_score = alphabeta(
    0, 0, float('-inf'), float('inf'), True, scores
```

)

print("Leaf Scores:", scores)

print("Best Score using Alpha-Beta Pruning:", best_score)

output:



24. PYTHON PROGRAM TO IMPLEMENT DECISION TREE

Aim

To write a Python program to implement a Decision Tree classification algorithm using the Scikit-learn library.

Algorithm

1. Start the program.
2. Import the required libraries.
3. Create a sample dataset.
4. Separate the input features and target values.
5. Create a Decision Tree classifier.
6. Train the model using the dataset.
7. Predict the output.
8. Display the prediction and accuracy.
9. Stop the program.

Python Code

Decision Tree Classification

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score

Sample dataset: [Study Hours, Attendance]

X = [

```
    [1, 50],
    [2, 60],
    [3, 70],
    [4, 80],
    [5, 90],
    [6, 95]
]

# 0 = Fail, 1 = Pass
y = [0, 0, 0, 1, 1, 1]

# Create and train the model
model = DecisionTreeClassifier(random_state=0)
model.fit(X, y)

# Predict using the training data
predictions = model.predict(X)

print("Actual Output:", y)
print("Predicted Output:", predictions)
print("Accuracy:", accuracy_score(y, predictions) * 100, "%")

# Predict for a new student
result = model.predict([[4, 85]])
print("New Student Prediction:",
      "Pass" if result[0] == 1 else "Fail")
```

Output:

```
result = model.predict([[4, 85]])
print("New Student Prediction:",
      "Pass" if result[0] == 1 else "Fail")
```

```
Actual Output: [0, 0, 0, 1, 1, 1]
Predicted Output: [0 0 0 1 1 1]
Accuracy: 100.0 %
New Student Prediction: Pass
```

25. PYTHON PROGRAM TO IMPLEMENT FEED FORWARD NEURAL NETWORK

Aim

To write a Python program to implement a Feed Forward Neural Network for classification using the Scikit-learn library.

Algorithm

1. Start the program.
2. Import the required libraries.
3. Create a sample dataset.
4. Initialize a Feed Forward Neural Network.
5. Train the network using the dataset.
6. Pass the input through the hidden layer and output layer.
7. Predict the output.
8. Display the predicted values and accuracy.
9. Stop the program.

Python Code

```
# Feed Forward Neural Network

from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Sample dataset: [Input 1, Input 2]
X = [
  [0, 0],
  [0, 1],
  [1, 0],
```

```
    [1, 1]
]

# XOR output
y = [0, 1, 1, 0]

# Create a neural network with one hidden layer
model = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation='tanh',
    solver='lbfgs',
    max_iter=5000,
    random_state=1
)

# Train the network
model.fit(X, y)

# Predict the output
predictions = model.predict(X)

print("Actual Output:", y)
print("Predicted Output:", predictions)
print("Accuracy:", accuracy_score(y, predictions) * 100, "%")

# Test a new input
result = model.predict([[0, 1]])
print("Output for [0, 1]:", result[0])
```

output:

```
Actual Output: [0, 1, 1, 0]
Predicted Output: [0 1 1 0]
Accuracy: 100.0 %
Output for [0, 1]: 1
```